Assignment -1

Student Name: Jayanaath S
UID: 23BCC70022
Branch: AIT-CSE
Section: 23AIT_KRG-1 A
Semester: 6
Date of Performance: 01/03/2026
Subject Name: Full Stack II
Subject Code: 23CSH-382

- **Aim:**
  The aim of this experiment is to design and implement a Daily Water Intake Tracker feature inside the EcoTrack React application, incorporating routing, protected authentication, state management, API integration, local storage persistence, and performance optimization using React best practices such as React.memo, useCallback, and useMemo.

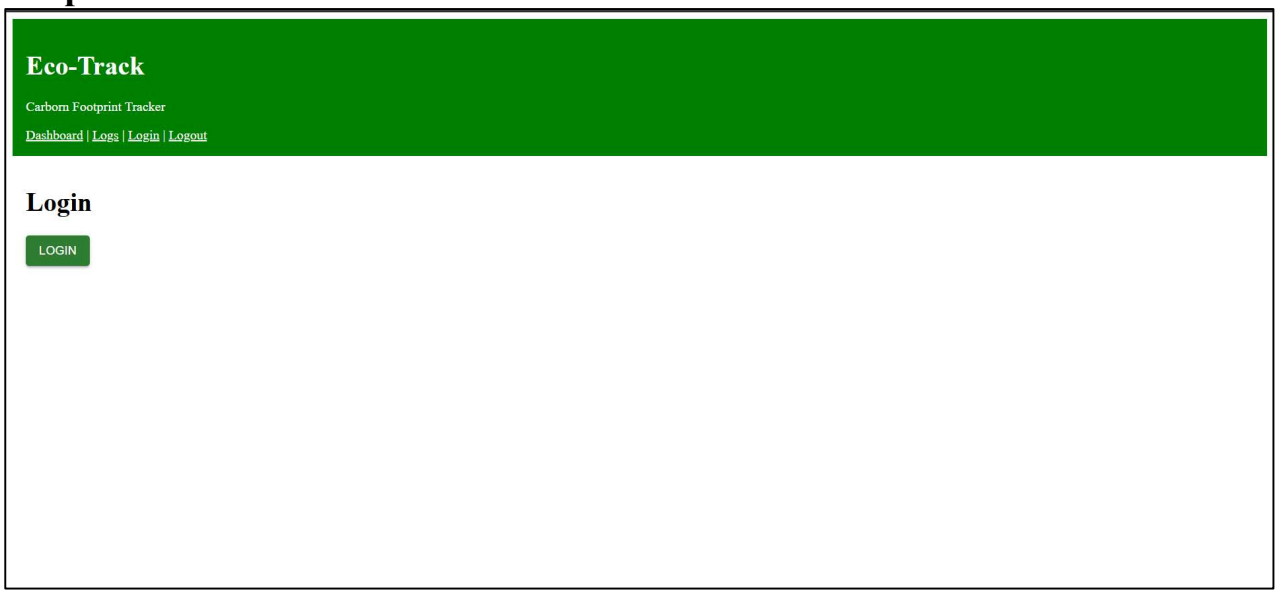
- **Objectives:**
  The main objectives of this experiment are as follows:
  1. To implement client-side routing using React Router.
  2. To create protected routes with authentication using localStorage.
  3. To manage component state using useState.
  4. To persist application data using useEffect and localStorage.
  5. To implement increment, decrement, and reset functionality.
  6. To display dynamic UI messages based on state conditions.
  7. To integrate a public API and handle loading and error states.
  8. To optimize component rendering using React.memo.
  9. To prevent unnecessary re-renders using useCallback/useMemo.
  10. To build a structured and maintainable dashboard-based React application.

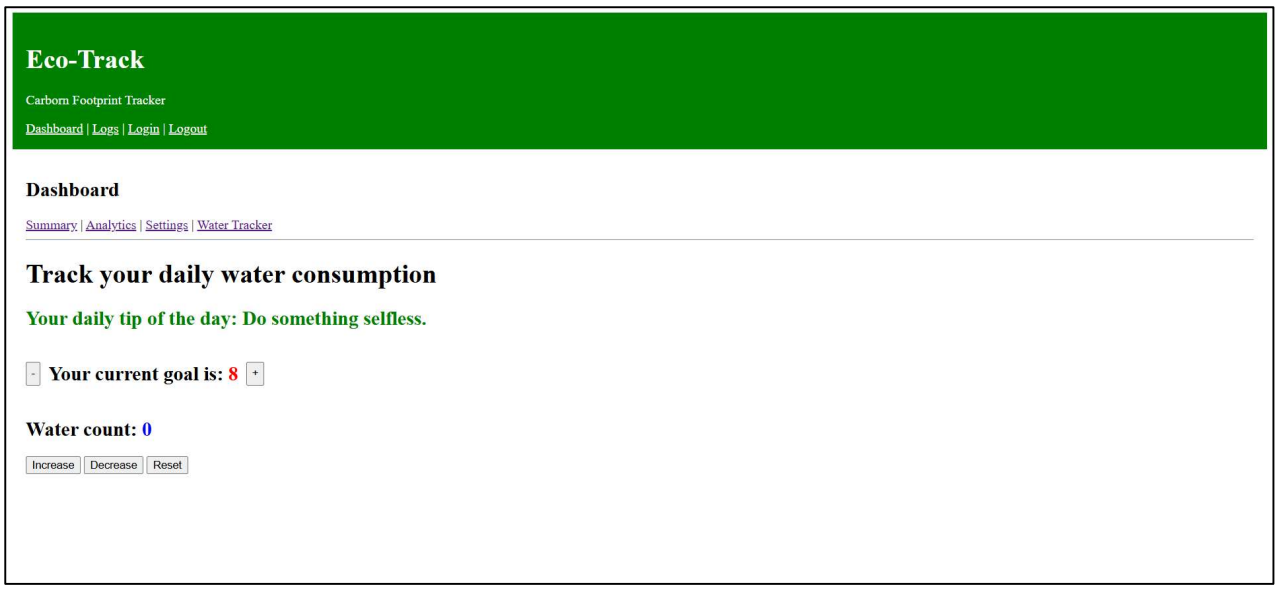
- **Implementation:**

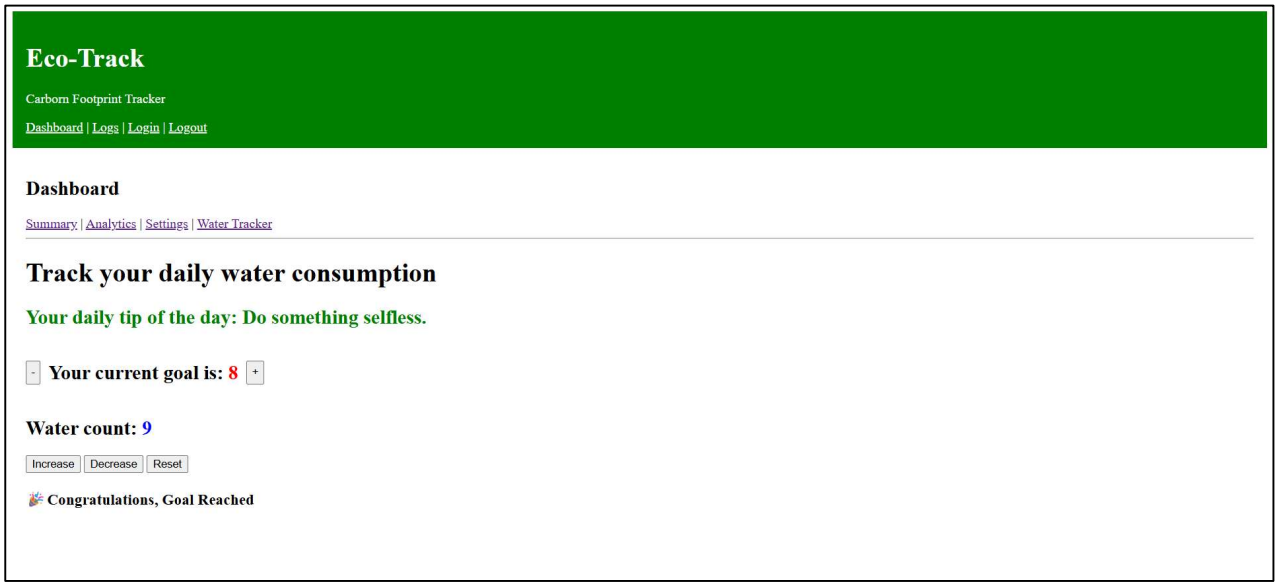
The Daily Water Tracker feature was implemented through the following structured steps:

1. **Routing & Authentication**
   - Created routes: /login, /dashboard, /dashboard/water.
   - Implemented a Protected Route that redirects unauthenticated users to login.
   - Fake login stores token in localStorage; logout removes it.
2. **State Management**
   - Used useState for count (default 0) and goal (default 8).
   - Implemented Add, Remove, Reset, and Save Goal functionality.
   - Displayed progress text and "Goal Reached" message when count ≥ goal.
3. **Local Storage Persistence**
   - Used useEffect to save count to localStorage.
   - Loaded saved value on page refresh.
4. **API Integration**
   - Fetched health tip from https://api.adviceslip.com/advice.
   - Implemented loading state and error handling.
   - Displayed "Today's Health Tip".
5. **Performance Optimization**
   - Created separate <CounterDisplay /> component.
   - Optimized using React.memo.
   - Used useCallback / useMemo to prevent unnecessary re-renders.

- **Output:**


Eco-Track
Carbon Footprint Tracker
Dashboard | Logs | Login | Logout
Login
LOGIN


Eco-Track
Carbon Footprint Tracker
Dashboard | Logs | Login | Logout
Dashboard
Summary | Analytics | Settings | Water Tracker
Track your daily water consumption
Your daily tip of the day: Do something selfless.
- Your current goal is: 8 +
Water count: 0
Increase Decrease Reset



- **Results:**

  The EcoTrack Daily Water Tracker feature was successfully implemented with routing, protected authentication, state persistence, API integration, and performance optimization.
  The application correctly:
  - Redirects unauthorized users.
  - Maintains state across refresh using localStorage.
  - Displays dynamic goal-based messages.
  - Fetches and displays health tips with proper error handling.
  - Prevents unnecessary re-renders using React.memo and useCallback.

  This experiment enhanced the application's usability, responsiveness, and performance while following modern React development standards.

- **Learning Outcomes:**

  After completing this experiment, I have learnt to:

  1. Implement protected routing using React Router.
  2. Manage state effectively using useState and useEffect.
  3. Persist application data using localStorage.
  4. Handle conditional UI rendering based on state.
  5. Integrate third-party APIs with loading and error handling.
  6. Optimize React applications using React.memo.
  7. Use useCallback and useMemo to prevent unnecessary re-renders.
  8. Structure dashboard-based applications properly.
  9. Improve user experience through state persistence.
  10. Apply performance optimization techniques in real-world React applications.